

ATENÇÃO:

**ESTE PROJETO É PARTE OBRIGATÓRIA DA AVALIAÇÃO
DA DISCIPLINA DE APC E TEM PESO 3 NA NOTA FINAL.
ALÉM DISSO, CASO O(A) ALUNO(A) NÃO OBTENHA
NOTA MÍNIMA 5,0 NO PROJETO, O(A) ALUNO(A)
RECEBERÁ MENÇÃO MI.**

Projeto Final

O objetivo deste projeto é que o aluno aprenda a usar **estruturas de dados do tipo matriz, subalgoritmos (funções), macros do pré-processador, bibliotecas diferentes das utilizadas até então, registros, arquivos, e variáveis globais de forma correta** em algoritmos, bem como implementá-los corretamente em programas.

Problema: Implementar um jogo (2D) **baseado** no jogo **Number Sums** (Figura 1) que está disponível para dispositivos móveis Android e IOS. O jogo começa com um grid 2D na forma de uma matriz, ou seja, com linhas e colunas compostos por números aleatórios de 1 a 9. Cada linha e cada coluna possui um somatório o qual é escrito no topo de cada coluna e na frente de cada linha. O jogador tem como objetivo apagar os números de modo que os números restantes nas linhas e colunas, uma vez somados, são exatamente o somatório indicado na frente da linha e/ou no topo da coluna. O jogador tem 2 vidas. As formas de obtenção de pontos, avanço nas fases, e fim do jogo, serão especificadas posteriormente.



Figura 1: Jogo inspiração - Number Sums 1

Detalhamento do problema:

As unidades de entrada deste projeto deverão ser arquivos em disco e o teclado. As unidades de saída serão o monitor de vídeo e um arquivo binário em disco para armazenamento do ranking do jogo.

Para facilitar o manuseio do programa, deverão ser dadas mensagens explicativas em alguns casos conforme indicado posteriormente nesta especificação.

Inicialmente deve ser mostrada uma tela de boas vindas com o **nome do jogo (escolhido pelo desenvolvedor)** onde é solicitado o nickname do jogador (Figura 2). Em seguida deve ser apresentada ao usuário a tela do menu principal, até que o usuário digite o número da opção que deseja e tecle <enter>. Conforme mostrado na Figura 3.

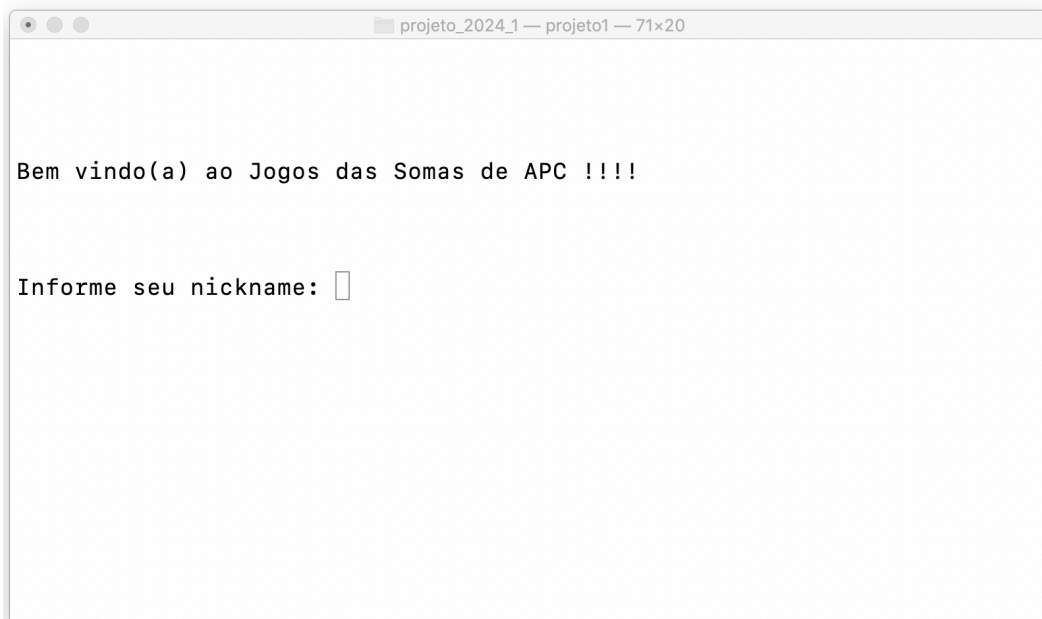


Figura 2: Tela de boas vindas



Figura 3: Menu Principal

A seguir será detalhado o quê deve ser feito em cada opção do Menu do Jogo.

1. JOGAR

O jogo sempre começa no modo iniciante e após passar por todas as fases do iniciante, passa-se automaticamente o modo intermediário e depois o avançado. São fornecidos 3 arquivos com os mapas de entrada de cada modo: `iniciante.txt`, `intermediario.txt` e `avancado.txt`. Caso o usuário escolha outro modo diferente do atual, o jogo deve automaticamente ajustar-se para mudar de modo (acessando assim outro arquivo de entrada). A troca para qualquer modo é permitida na opção de CONFIGURACOES, tanto de níveis mais fáceis para mais difíceis como o contrário.

Tela inicial da Opção 1. JOGAR:

O jogo deve mostrar a matriz lida do respectivo arquivo texto, bem como os somatórios de cada linha e coluna, também obtidos do arquivo. A Figura 4 ilustra essa tela para o modo iniciante.

Objetivo do jogo:

O jogo sempre começa com 5 vidas. O objetivo do jogo é apagar os elementos das linhas e colunas de modo que os elementos restantes na tela, uma vez somando são exatamente os valores mostrados no topo das colunas e na frente das linhas. Para apagar um elemento o jogador deve informar a linha e a coluna do número a ser apagado. No modo iniciante a matriz do jogo é 4x4, no modo intermediário a matriz é 6x6 e no modo avançado a matriz é 7x7. Para o jogador as linhas e colunas iniciam sempre em 1 (lembrando que no programa elas iniciam em 0).

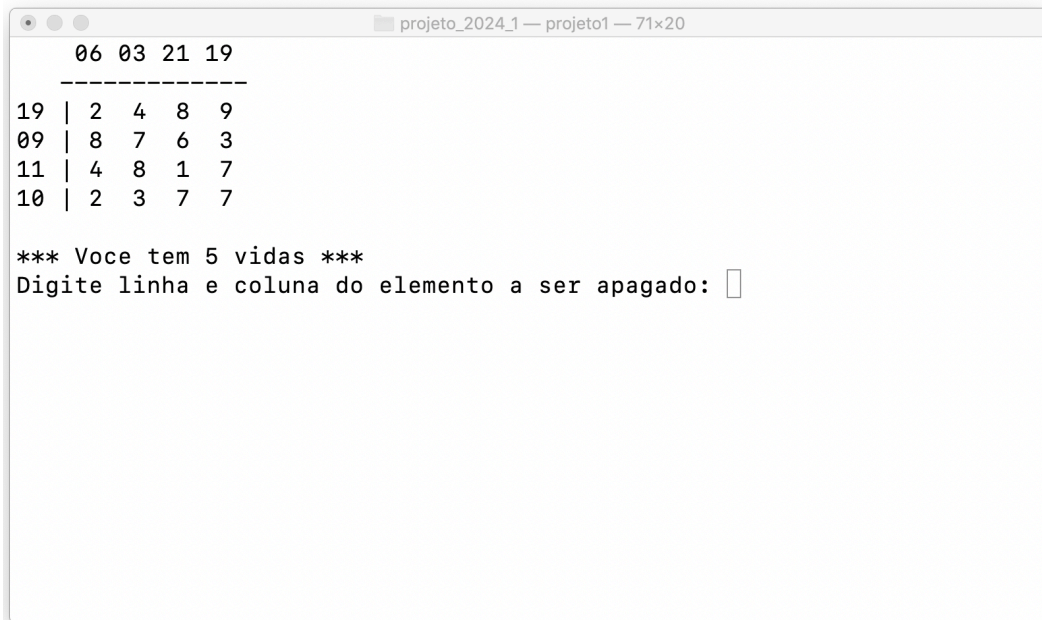


Figura 4: Tela inicial do modo iniciante

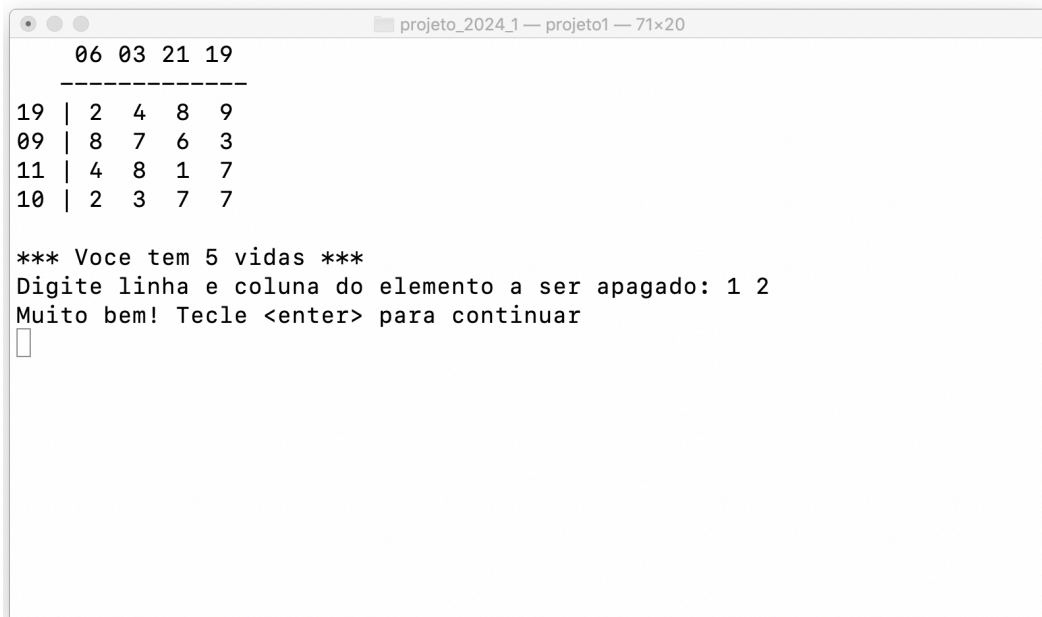


Figura 5: Jogador acertou o elemento a ser apagado

```
projeto_2024_1 — projeto1 — 71x20

06 03 21 19
-----
| 2      8  9
09 | 8  7  6  3
11 | 4  8  1  7
10 | 2  3  7  7

*** Voce tem 4 vidas ***
Digite linha e coluna do elemento a ser apagado: 
```

Figura 6: Matriz atualizada após elemento apagado

```
projeto_2024_1 — projeto1 — 71x20

06 03 21 19
-----
19 | 2  4  8  9
09 | 8  7  6  3
11 | 4  8  1  7
10 | 2  3  7  7

*** Voce tem 4 vidas ***
Digite linha e coluna do elemento a ser apagado: 1 1
Errou, tecle <enter> e tente novamente

```

Figura 7: Jogador tentou apagar um elemento que deve permanecer na matriz

```
projeto_2024_1 — projeto1 — 71x20

06      21 19
-----
| 2      8 9
|      6 3
11 | 4      1 7
10 | 2 3 7 7

*** Voce tem 3 vidas ***
Digite linha e coluna do elemento a ser apagado: 3 3
Muito bem! Tecle <enter> para continuar
Parabens! Voce fechou a linha 3. Tecler <enter>!

Parabens! Voce fechou a coluna 3. Tecler <enter>!

```

Figura 8: Jogador fechou uma linha e uma coluna

```
projeto_2024_1 — projeto1 — 71x20

19
-----
| 2      8 9
|      6 3
| 4      7
10 |      3 7 7

*** Voce tem 3 vidas ***
Digite linha e coluna do elemento a ser apagado: 4 4
Muito bem! Tecle <enter> para continuar
Parabens! Voce fechou a linha 4. Tecler <enter>!

Parabens! Voce fechou a coluna 4. Tecler <enter>!

```

Figura 9: Jogador fechou uma linha e uma coluna

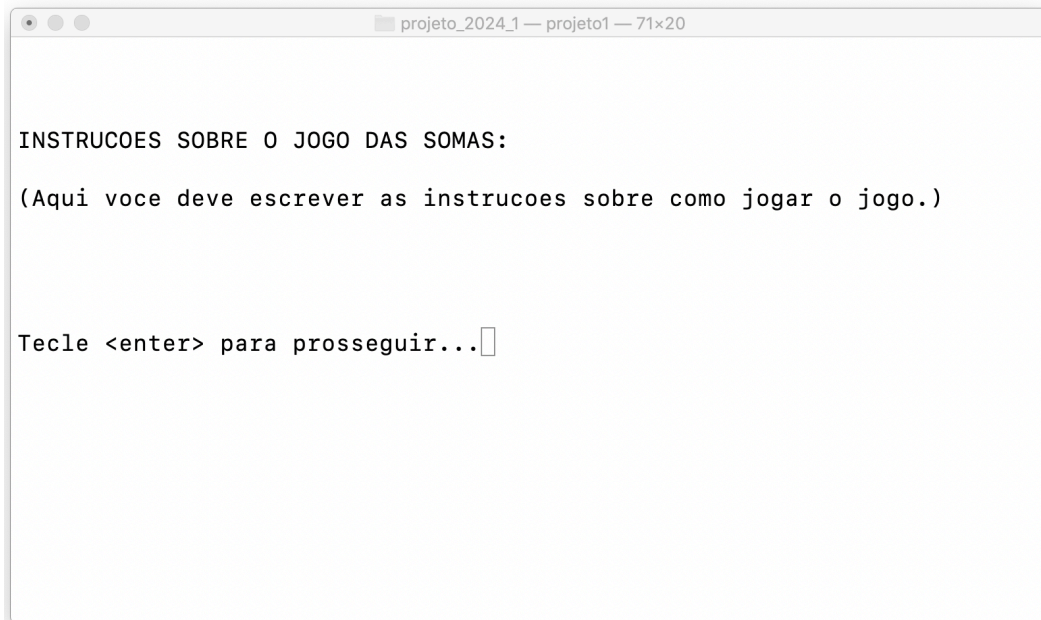


Figura 10: Tela para mostrar as instruções do jogo

Após digitar a linha e coluna, o programa deve verificar na matriz espelho (explicada posteriormente) se o elemento pode ser apagado, ou seja, se a matriz espelho contém o caracter zero (0) naquela posição. Caso o jogador tenha acertado, deve ser mostrada a mensagem de acordo com a Figura 5 e em seguida a matriz do jogo deve ser atualizada na tela sem o elemento apagado (veja Figura 6). Você pode observar que esta Figura 6 está com o número de vidas alterado, provavelmente por que antes o jogador escolheu um número errado. Ao acertar o número a ser apagado, a quantidades de vidas permanece a mesma.

Caso o jogador digite uma posição onde está um número que não deve ser apagado, o programa deve mostrar uma mensagem de erro, conforme mostra a Figura 7.

Assim que o jogador apagar um elemento correto, o programa deve verificar se o jogador não “fechou” uma linha e/ou uma coluna com esta ação. “Fechar” significa que mais nenhum dos elementos da linha ou da coluna em questão deve ser apagado, ou seja, os elementos restantes somados formam exatamente o valor indicado no topo da coluna ou na frente da linha. Quando isto acontecer o programa deve dar uma mensagem ao jogador conforme indicam as Figuras 8 e 9. Lembrando que após a eliminação de um elemento podem ocorrer as seguintes situações:

- Apagar um elemento que ainda não “fecha” a linha nem a coluna
- Apagar um elemento que “fecha” somente uma linha
- Apagar um elemento que “fecha” somente uma coluna
- Apagar um elemento que “fecha” uma linha e uma coluna

Assim que uma linha e/ou coluna for “fechada” o respectivo somatório deve desaparecer da tela, mas os números que formam aquele somatório devem permanecer. Quando todas as linhas e colunas forem “fechadas” o jogo deve dar uma mensagem conforme mostrado na Figura 19 e retornar ao menu principal. Neste momento também deve ser gravada a pontuação e o nickname do jogador no arquivo `ranking.bin`. Caso ele não exista na primeira vez, ele deve ser criado. Toda vez que for fazer essa gravação o programa deve verificar se aquele nickname já está no ranking, e caso estiver, deve APENAS adicionar a pontuação correspondente à pontuação atual, caso contrário deve adicionar a pontuação e o nickname ao ranking. Lembrando que essa busca deve ser feita em memória RAM, ou seja, o arquivo `ranking.bin` precisa ser carregado para a memória (vetor de struct) para verificação se o nickname já se encontra no ranking.

As gravações no arquivo `ranking.bin` devem ser feitas sempre em struct ou vetor de struct:

```
struct dadosjogador {  
    char nickname[20];  
    int pontuacao;  
};
```

Cada partida no modo INICIANTE vale 50 pontos, no modo INTERMEDIARIO vale 100 pontos e no modo AVANÇADO vale 200 pontos.

Ao perder as 5 vidas, deve ser dada uma mensagem informando que o jogador perdeu, e assim retornar ao menu principal. Caso o jogador escolha novamente a opção para jogar, a **matriz original com todos os elementos** deve aparecer na tela. Enquanto o jogador não encerrar o programa, na opção 1. JOGAR deve sempre ser mostrada a matriz original da última fase que ele não passou, dentro daquele modo. Porém, no caso do jogador alterar o modo, o jogo deve sempre iniciar com a primeira matriz do modo escolhido.

Quando o jogador termina todas as fases (matrizes) de um modo, o jogo muda automaticamente para as matrizes do próximo modo, sem informar nada ao usuário.

Quando o jogador vencer a última matriz do modo avançado, o programa deve informar ao jogador a mensagem PARABÉNS!!! VOCÊ TERMINOU (ZEROU) O JOGO!

Organização dos arquivos .txt:

Os três arquivos estão organizados da mesma forma. Os dados de cada partida estão separados por um caractere asterisco (*) dentro do arquivo .txt. Inicialmente estão os dados da matriz inicial que contém os números que deverão compor o tabuleiro a ser jogado. Em seguida existem 2 linhas que representam, respectivamente, a soma de cada coluna da matriz e a soma de cada linha da matriz (se 2 em 2 caracteres). Após isso, há uma matriz composta apenas por zeros (0) e uns (1), chamada de matriz “espelho” que representa os números que devem permanecer ou não na matriz do jogo de modo que restem somente os números que uma vez somados representem o mesmo somatório inicial das linhas e colunas lidos do arquivo .txt. Onde aparece zero (0) na matriz espelho o número da matriz do jogo naquela posição deve ser apagado e onde aparece um (1) o número correspondente deve ser mantido em tela.

ATENÇÃO: após cada atualização, você deve limpar a tela e mostrar a matriz na mesma posição anterior.

2. INSTRUCOES

A opção INSTRUCOES deve mostrar um texto explicativo sobre o funcionamento do jogo, as ações que o jogador pode fazer, o objetivo do jogo, pontuação, níveis, evolução, vidas, formas de encerramento do jogo, etc. A Figura 10 mostra um exemplo ainda sem as instruções escritas. Ao final uma mensagem para prosseguir deve ser mostrada até que o usuário pressione <enter>.

3. CONFIGURACOES

Ao escolher a opção 3. CONFIGURACOES, deve ser mostrado ao jogador um novo menu (Figura 11).

A seguir serão detalhadas as opções do Menu CONFIGURACOES:

(MENU CONFIGURACOES) 1. ZERAR RANKING: nesta opção o programa deve zerar completamente o arquivo ranking.bin, ou seja, deve recriá-lo vazio. É necessário perguntar ao usuário se ele confirma a operação. Caso o usuário digitar “N” ou “n” nenhuma ação deve ser executada no arquivo ranking.bin. A Figura 12 ilustra esta opção ZERAR RANKING.

(MENU CONFIGURACOES) 2. MODO DE DIFICULDADE: nesta opção deve ser mostrado ao jogador um novo menu conforme a Figura 13. Cada uma das opções de 1 a 3 irá alterar o modo atual do jogo (que sempre inicia como “Iniciante”) para qualquer um dos modos escolhidos. Inclusive se o jogador estiver nos modos 2 ou 3 ele pode escolher voltar para o modo 1. Após cada alteração de modo, deve ser dada uma mensagem indicando qual modo está ativo (Figura 14). Na opção 4, o programa deve retornar ao menu anterior que é o de CONFIGURACOES.

(MENU CONFIGURACOES) VOLTAR AO MENU PRINCIPAL: nesta opção o programa deve retornar para o menu principal/inicial do jogo.

Aqui encerramos a descrição do sub-menu CONFIGURACOES e prosseguimos com a explicação do menu principal

4. RANKING

Ao escolher a opção 4. RANKING, deve ser mostrado, na mesma tela, abaixo do menu, o ranking atual (que pode estar vazio ou não), ordenado de forma decrescente (ou seja, do maior para o menor) por ordem de pontuação (Figura 15 e Figura 16).

5. SAIR

Ao escolher a opção 5. SAIR, o programa deve ser encerrado.



Figura 11: Tela do Menu Configurações

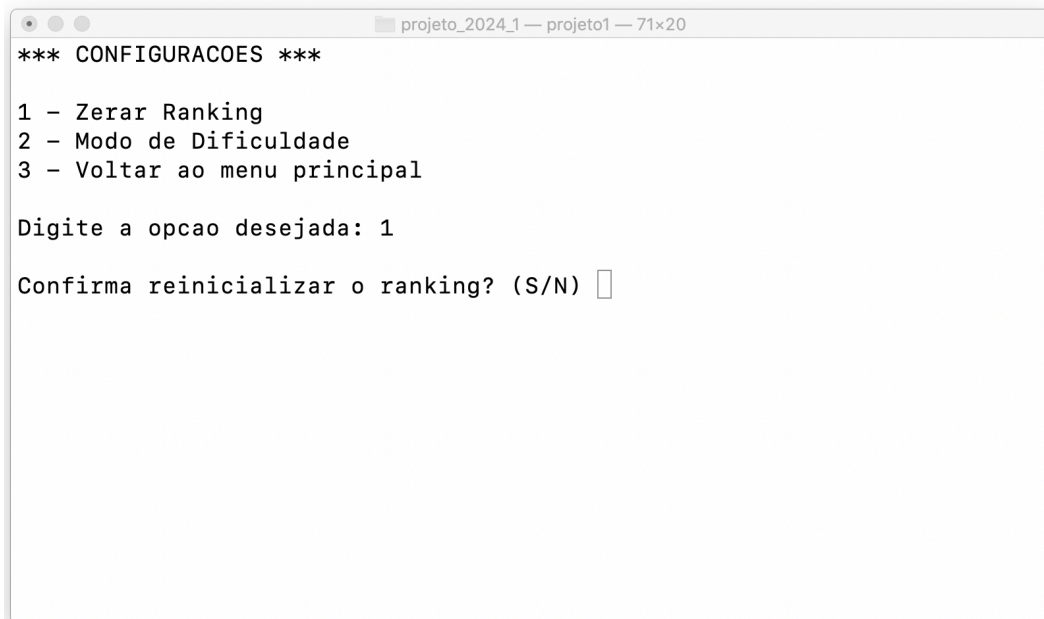


Figura 12: Tela de confirmação para Zerar Ranking

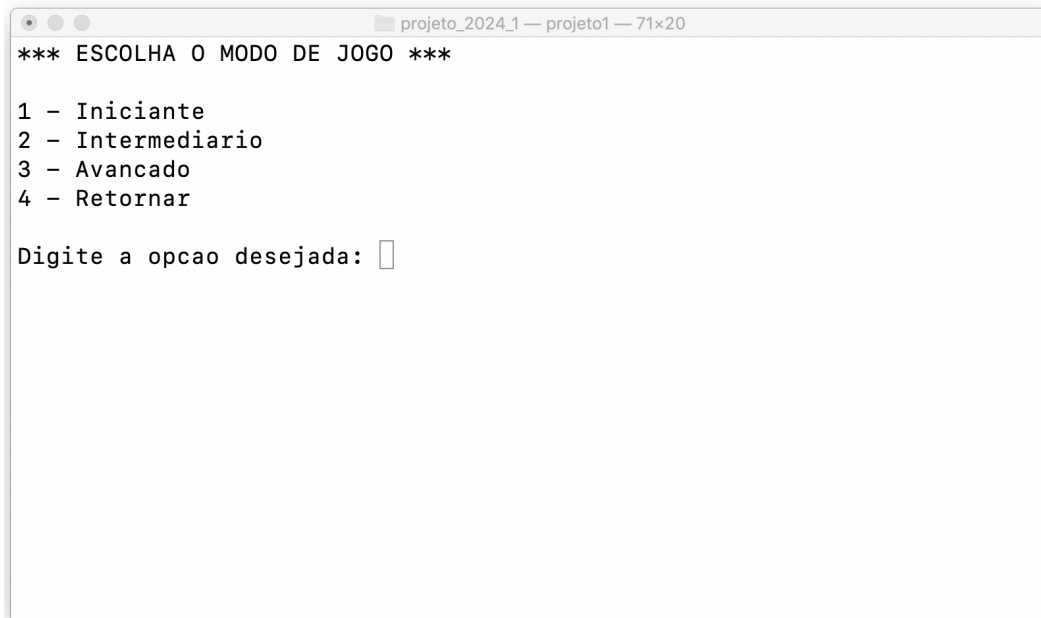


Figura 13: Tela do Menu para escolha do Modo de Jogo

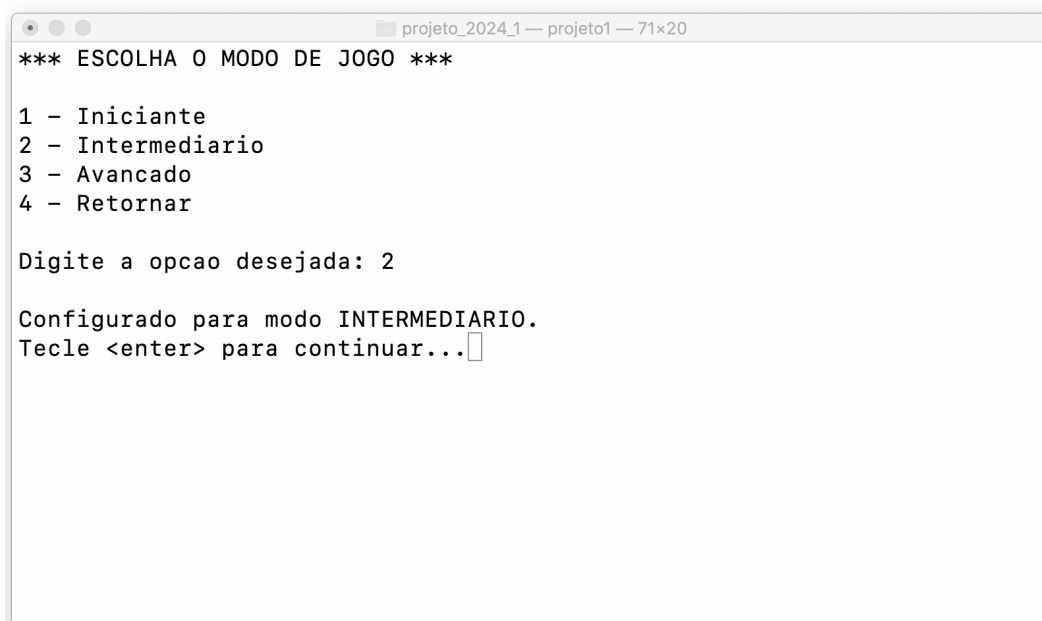


Figura 14: Tele de configuração para o modo intermediário

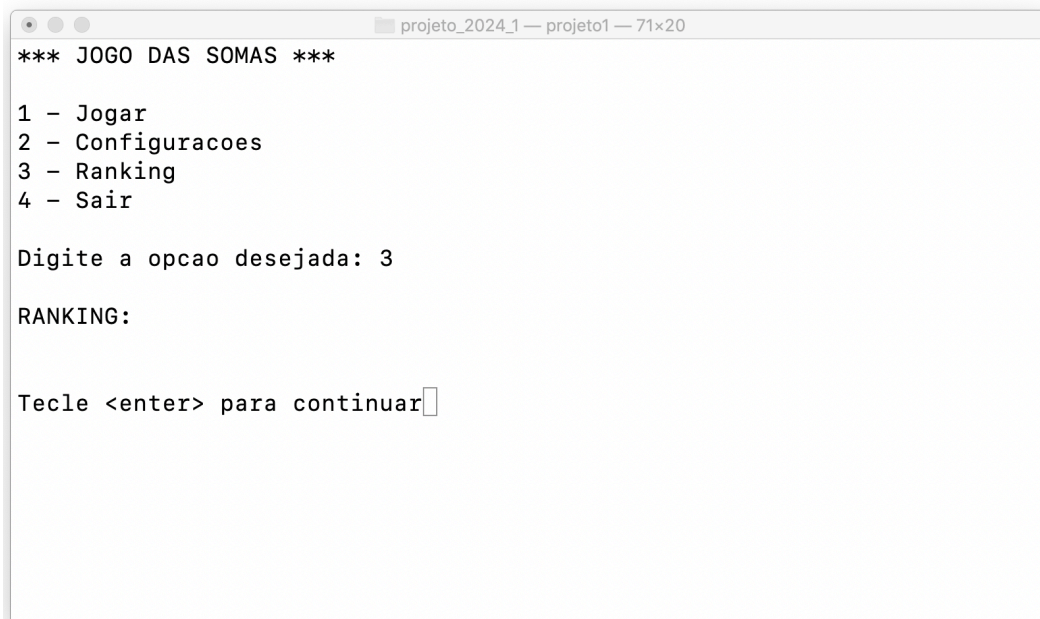


Figura 15: Tela mostrando ranking vazio

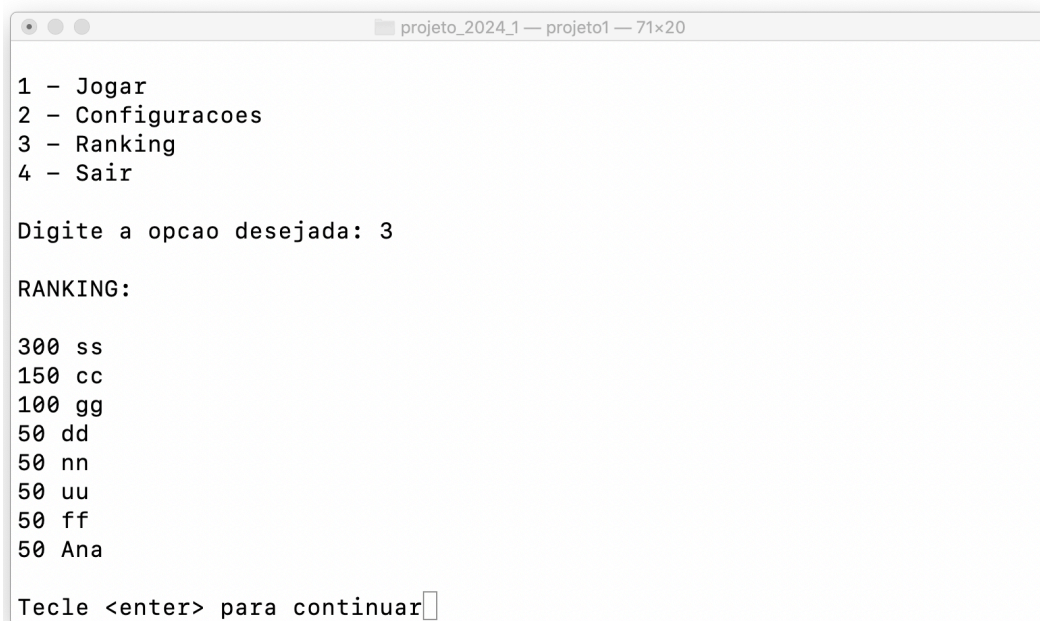


Figura 16: Tela mostrando ranking em ordem decrescente de pontuação

A seguir são apresentadas mais algumas telas ilustrativas do jogo:

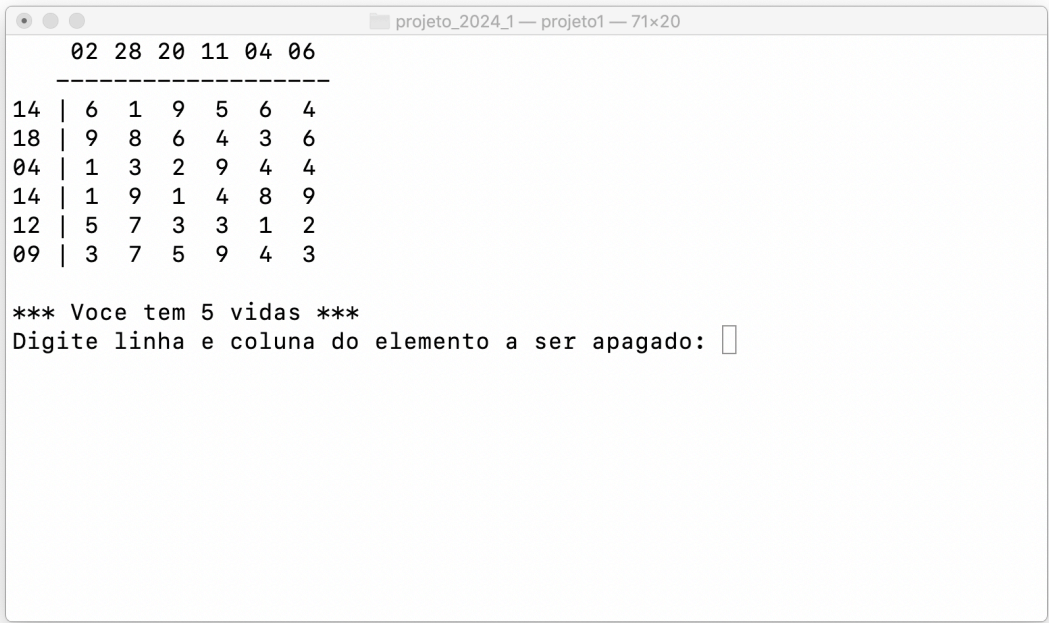


Figura 17: Tela inicial do modo intermediário

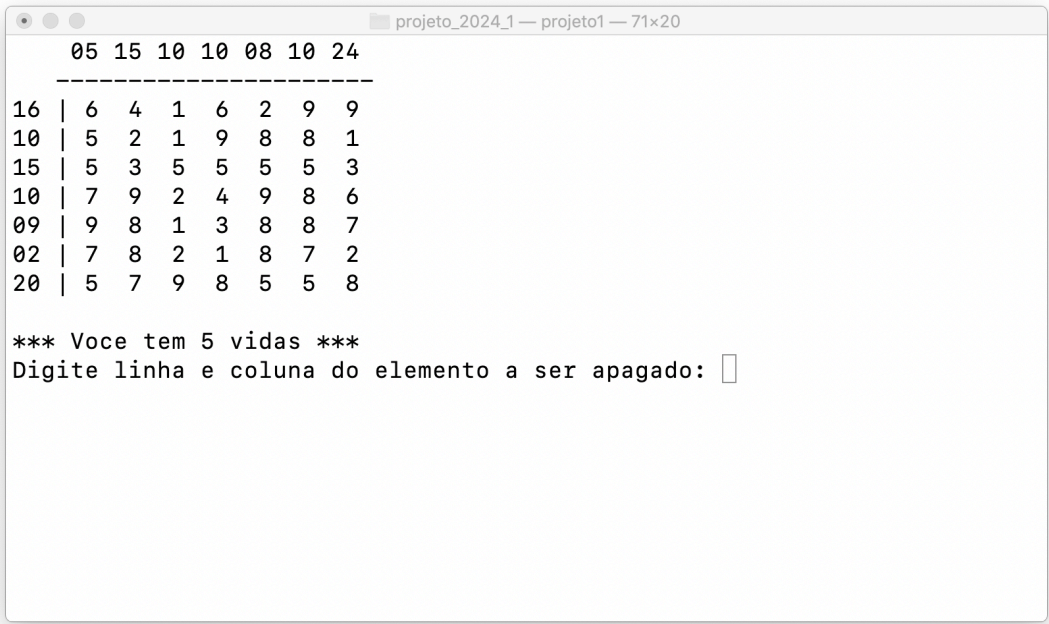


Figura 18: Tela inicial do modo avançado

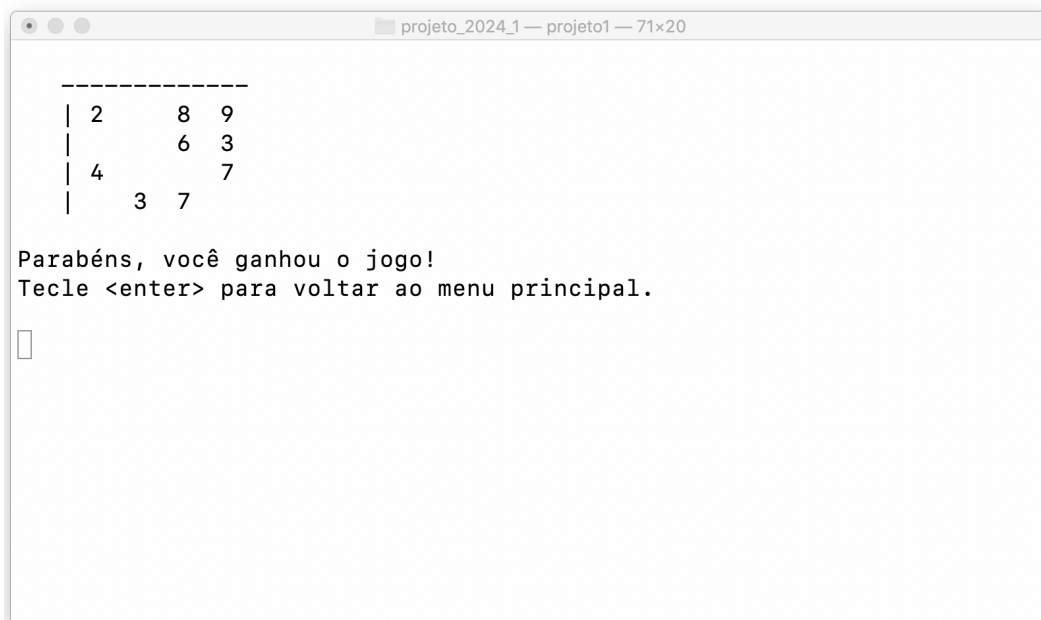


Figura 19: Tela ao ganhar a partida

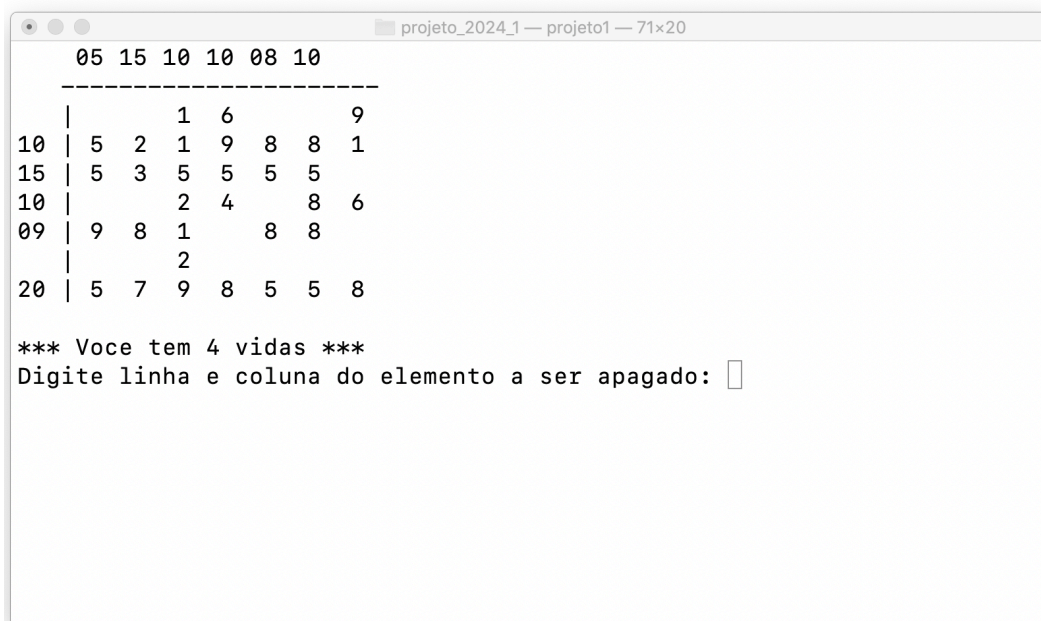


Figura 20: Tela do modo avançado em andamento

(veja o APÊNDICE com detalhes de implementação nas próximas páginas!)

APÊNDICE:

Neste projeto você DEVE modularizar o seu programa criando funções. Conforme já dito em sala de aula, funções tornam o seu programa mais legível, mais fácil de ser entendido, e mais curto, já que evita a duplicação de código para uma mesma ação. Será de total responsabilidade do programador decidir quais funções serão criadas, definindo seus tipos, conteúdo e funcionalidade.

Seguem algumas **sugestões** de funções:

- Funções para mostrar cada menu na tela (são 3 menus!);
- Função para ler a matriz do jogo do arquivo para a memória (a leitura das strings contendo a soma das linhas e colunas podem ser feita aqui, por exemplo);
- Função para ler a matriz espelho do arquivo para a memória;
- Função para executar o jogo;
- Função para mostrar a matriz do jogo na tela juntamente com os somatórios das linhas e colunas;
- Função para ler o ranking do arquivo, ordenar e mostrar o ranking na tela (lembrando que se desejar você pode guardar o ranking ordenado e fazer esta ordenação no momento em que adicionar um novo jogador nele);
- Função para gravar no ranking um novo jogador e sua pontuação;
- Função para zerar o ranking;
- Função para mostrar as instruções (Opção 3 do menu principal);
- Função para restaurar as matrizes originais quando o jogador perder todas as vidas e escolher jogar novamente;
- etc...

Novo cabeçalho, Macros e variáveis globais: neste trabalho, será necessário o uso de macros do pré-processador, o cabeçalho *stdlib.h* e **variáveis globais**.

1) Das macros do pré-processador

As diretivas do pré-processador, isto é, os comandos iniciados com o caractere # que incluímos no início de nossos arquivos de código-fonte são meios de realizar modificações em um programa em tempo de compilação. Cada uma das diretivas instrui o pré-processador a realizar uma modificação no código-fonte, antes que o compilador vá, de fato, gerar o arquivo executável de um programa.

Até agora, conhecemos apenas a diretiva *#include*. Esta diretiva diz ao pré-processador para, literalmente, substituir sua declaração por todo o código que está dentro do arquivo especificado.

Neste trabalho, utilizaremos uma nova diretiva: o *#define*. Esta diretiva diz ao pré-processador para, literalmente, substituir a **macro definida** em todos os lugares que ele aparecer no código-fonte por sua **expansão**. **Por exemplo:** no código abaixo, será compilada a instrução *printf("%d\n", 50)* e não a instrução *printf("%d\n", MAX_JOGOS)*. Além do mais, o compilador não saberia como compilar esta última instrução, já que não declaramos nenhuma variável com o identificador *MAX_JOGOS*. Vejam:

```
#define MAX_JOGOS 50
...
printf("%d\n", MAX_JOGOS);
...
```

Por ser, tradicionalmente, uma boa prática de programação, definimos macros utilizando caracteres maiúsculos, para que seja fácil identificá-los em um código-fonte. Isto é, ao visualizarmos identificadores em que estão presentes apenas caracteres maiúsculos, podemos nos assegurar de que este identificador é uma macro, um *#define*.

É possível combinar diretivas utilizando diretivas condicionais, exemplo:

```
#ifdef _WIN32
    #define CLEAR "cls"
#else
    #define CLEAR "clear"
#endif
```

As diretivas acima definem uma macro com um comando utilizado para limpar a tela, de acordo com o sistema operacional em que o código-fonte for compilado. Quando um compilador está sendo executado no Windows, a macro `_WIN32` fica definida no ambiente e pode ser utilizada em diretivas do pré-processador. O trecho acima exemplifica uma maneira de escrever código que compila e executa corretamente em diferentes sistemas operacionais.

Observação: No Windows, o comando de terminal `"cls"` de fato limpa todo o terminal. No Linux, o comando equivalente `"clear"` apenas coloca algumas quebras de linha e sobe a tela.

2) **switch** – Uma estrutura condicional extremamente recomendada para a implementação de **MENUS** é a estrutura `switch`. Esta estrutura é apenas uma maneira simplificada de escrever uma sequência `if-else if-else if-...-else if-else`, com a diferença de que o teste condicional realizado apenas verifica se um valor inteiro bate com uma das constantes selecionadas. Exemplo para o menu principal do seu jogo:

```
switch (menuMain()) {
    case MENUMAIN_JOGAR: jogar(); break;
    case MENUMAIN_INSTRUcoes: instrucoes(); break;
    case MENUMAIN_CONFIGURACOES: configuracoes(); break;
    case MENUMAIN_RANKING: ranking(); break;
    case MENUMAIN_SAIR: sair(); break;
    default:
        exit(EXIT_FAILURE);
}
```

Uma sequência equivalente seria:

```
MenuMainOpt opt = menuMain();
if (opt == MENUMAIN_JOGAR)
    jogar();
else if (opt == MENUMAIN_CONFIGURACOES)
    configuracoes();
else if (opt == MENUMAIN_INSTRUcoes)
    instrucoes();
else if (opt == MENUMAIN_RANKING)
    ranking();
```

```

else if (opt == MENUMAIN_SAIR)
    sair();
else
    exit(EXIT_FAILURE);

```

3) Das variáveis globais

No início do curso, vimos que variáveis globais podem confundir o programador ao manipular seus dados e decidimos não utilizá-las. No entanto, em alguns casos, a melhor opção pode ser utilizar variáveis globais, mas o cuidado deve ser mantido. O objetivo desta seção é mostrar formas de se detectar em quais situações uma variável deve ser global. A seguir são indicadas algumas variáveis que podem ser globais neste trabalho.

Abaixo segue um **exemplo**, das variáveis globais que a professora usou no programa exemplo do jogo (mostrado em aula).

ATENÇÃO: você deve criar as suas próprias variáveis globais, conforme achar necessário. **Não é obrigatório** utilizar as variáveis abaixo.

```

FILE* fp;
char nickname[20], arquivo[20];
int pontuacao=0, modo=1,
Int tam_matriz, perdeu=0;
char matriz[10][10];
char matrizEspelho[10][10];
char SomaCol[20];
char SomaLin[20];
int VetContLin[10];
int VetContCol[10];

char matriz_orig[10][10];
int VetContLin_orig[10];
int VetContCol_orig[10];
char SomaCol_orig[20];
char SomaLin_orig[20];

```

É possível que outras variáveis também precisem ser globais neste programa. Fica a cargo do programador tomar as decisões sobre quais variáveis devem ser globais. **Na correção do trabalho, estas decisões serão avaliadas.**

4) Outras observações

Função system() - utilize a função `system()` do cabeçalho “`stdlib.h`” para limpar o terminal, a cada vez que o programa for exibir uma nova tela. Utilizando a dica do item 1 do Apêndice, basta realizar a chamada `system(CLEAR)`;

Crie variáveis e nomes de funções mnemônicos!!!

Utilize variáveis globais com cautela conforme explicado acima.

Nas funções, utilize **passagem de parâmetros por valor e por referência**, conforme necessário. Lembrando que as funções podem não receber nenhum parâmetro, como por exemplo, quando estão trabalhando apenas com variáveis globais. Além disso, podem ser do tipo void caso não retornem nada.

Faça diversos testes no seu jogo, jogando é claro. Bom trabalho e boa diversão!

Observações Gerais:

1. Incluir cabeçalho como comentário, informando sua matrícula, seu nome completo, e qualquer outra informação que você julgar necessário informar sobre o seu código.
2. A data de entrega do programa é: **16/09/2024 (2a-feira) até às 23:55 hs** pelo link disponível no Moodle.